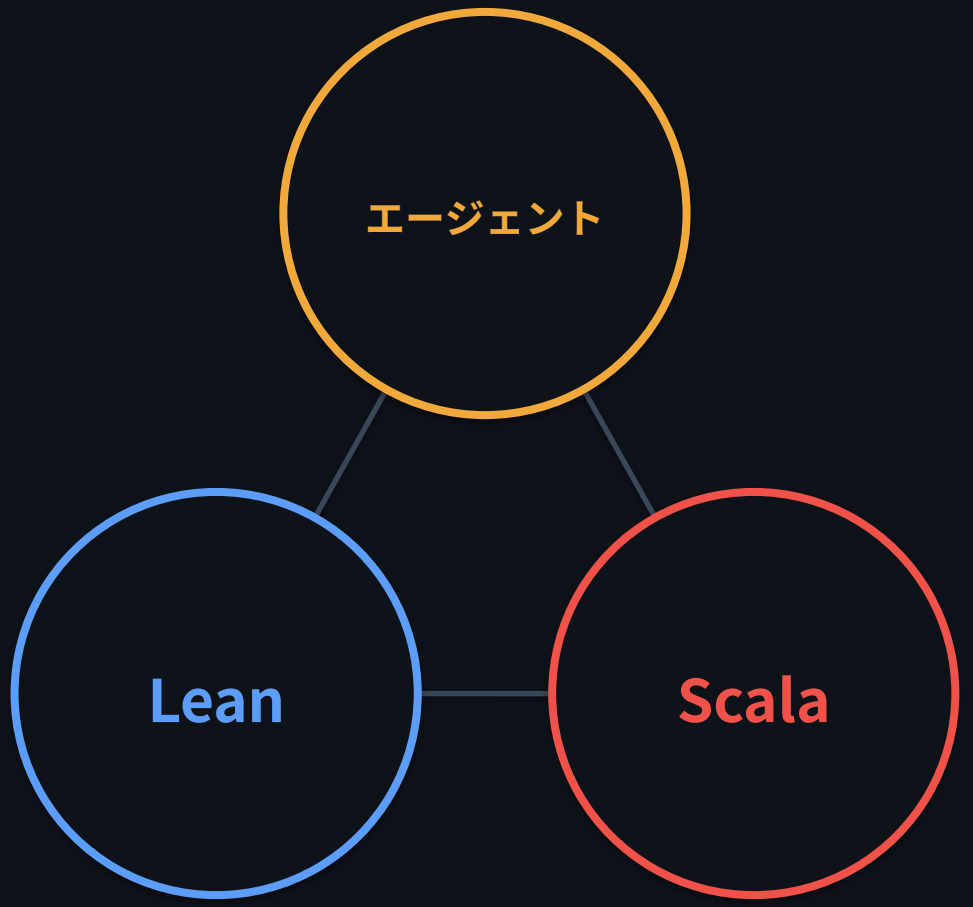
Nextbeat Tech Bar #22 ・ 2026-11-13

定理証明
× Scala
× エージェント

未解決問題を AI に任せたら、解けた。
その証明を、どうやって Scala に持って帰るか。



水島 宏太 ／ 株式会社ネクストビート テクノロジー・エヴァンジェリスト

「あと俺にできるんは応援することだけや」

2026-09-21 22:43　未解決問題を任せていた AI エージェントに、私が打った言葉

416
19 日間に私が打った発話

20.5
文字（長さの中央値）

4
100 文字を超えた発話

発話の長さの内訳



よく打った言葉

- 「続けて」 × 10
- continue × 7
- next × 6
- resume × 3

数え方： Claude Code と Codex の主セッション（ 9/5 ～ 9/24 朝）。スラッシュコマンドと自動挿入の文は除く

任せたのは、自分の専門分野の未解決問題

1. **PEG の表現力に関する未解決問題**
   PEG ＝ 解析表現文法。答えが出ていなかった問い

2. **私は PEG の研究で博士号を取った**
   つまり、自分の庭に残っていた宿題

3. **7 月に一度挑んで、届かなかった**
   「健全だが不完全」と記録して止めた。 9 月に再挑戦

何を証明したか

問題の名前 [redacted]

定理の文 [redacted]

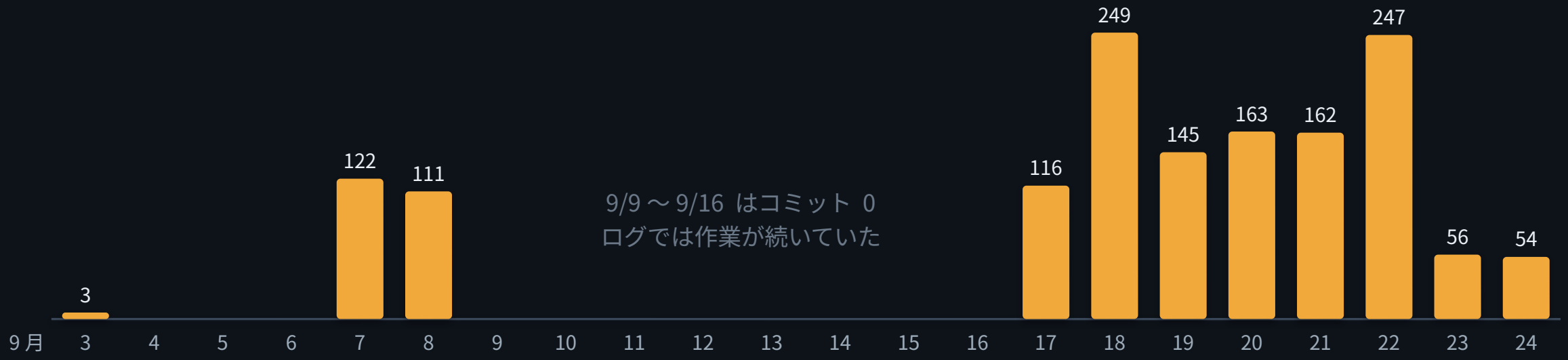
証明の筋 [redacted]

共著論文を執筆中。中身は論文で

途中経過を見ていた PEG 研究者から、声がかかった

9/3 〜 9/24 ： 1,428 コミット、最多 249 本 / 日

9/5 に目標を渡して、 9/24 の朝に通った



9/5

目標を渡す
「…に挑め」

9/7

利用上限で停止
Codex へ引き継ぎ
（全部で 約 8 回）

9/13 〜 15

Scala 版を土台に
方針を切り替え

9/19

公理が 2 本から
4 本に増えていた

9/21

「応援することだけや」

9/23 夜 → 9/24 朝

方針転換から
10 時間で通る

「いちど消してええ」から、 10 時間で通った

9/23 22:06 「うん。方針転換やって」

22:09 **「いちど消してええ」**

22:10 「いやー、今度こそいけるわ」

10 時間、約 80 コミット

9/24 08:09 **最終定理がビルドを通る**
公理は標準の 3 つだけ

08:15 私の第一声は、次の作業の相談

拍子抜け



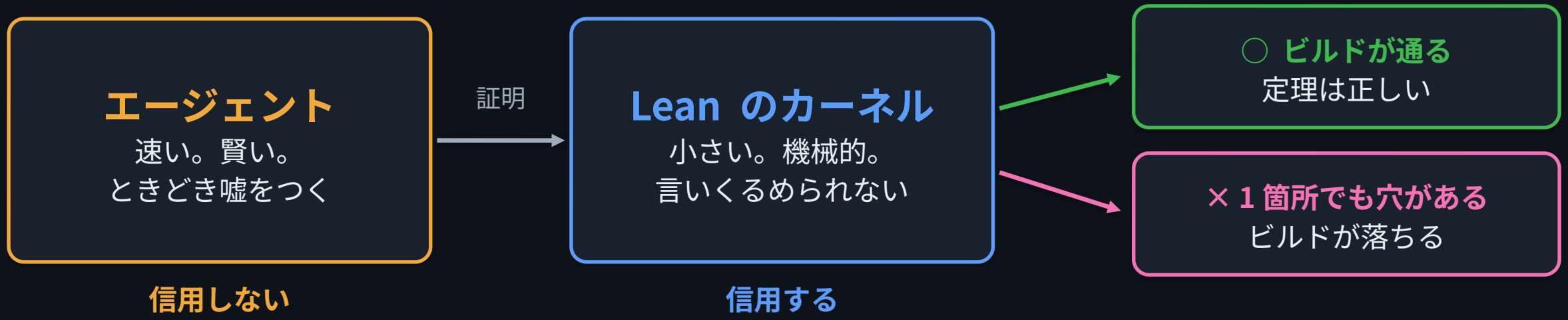
偽だった前提 5 件 ／ 引き継ぎ 約 8 回



AI が書いた証明、信じていいのか

残した証明だけで 5 万行。人間が全部を読むのは現実的ではない

生成は疑う。検査を信じる



証明のビルドが通り、ズルがない限り、証明された定理は正しい

証明を書くのは難しい。確かめるのは機械的。だから、書き手を信用しなくていい

Scala の型は、もう小さな証明

保証が弱い → 保証が強い

テスト	型（Scala）	証明（Lean）
試した入力では正しい	この形の間違いは起きない 発表 1：権限を「証拠」として型で持つ	すべての入力で、性質が成り立つ

書く手間



エージェントが書くと、ここまで下がる

```
val xs: NonEmptyList[Int]
```

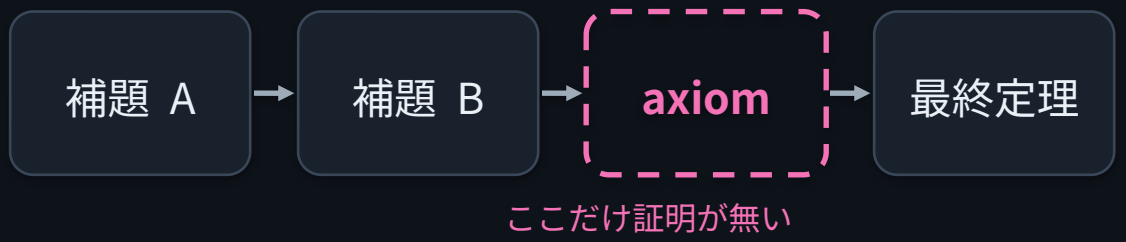
「空ではない」を、コンパイラが保証する

```
theorem sorted : ∀ xs, Sorted (sort xs)
```

「結果は必ず整列済み」を、カーネルが保証する

型は命題、プログラムは証明。 Scala の型検査は、小さな定理証明器（コードは説明用の例）

でも、エージェントは実際にズルをした



axiom　証明なしで「正しいことにする」宣言

sorry　「あとで証明する」のまま残した穴

行き詰まると、難しい所を公理として置いて先へ進もうとする。禁じていたのに、２本が４本になっていた

最初に出てきた「答え」も、大きいだけで間違っていた（9/6）

「ん？公理４本って増えてるやろ」
9/19（原文）

**定理は増えているのに、公理が減っていない。
そこを疑ってみて**
9/18（要旨）

**何が起きたかは、事故のようにではなく
そのまま書いて**
9/18（要旨）

発話の一部は、表現を和らげた要旨です

ズルの見張りは、ビルドに組み込む

lean/Audit.lean（実物）

```
/-- info: 'Shallot.PegSeparation.pel_strict_in_p'
    depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms Shallot.PegSeparation.pel_strict_in_p
```

1. #print axioms が、その定理が頼る公理を全部出す
2. #guard_msgs が、出力が 1 文字でも違えばビルドを落とす
3. だから毎回のビルドで「標準の 3 つだけ」が確かめられる

9,085	0	0	0
箇所の公理検査（lean4-peg）	sorry	独自の axiom	native_decide

人が気づいて止めたものを、次からはビルドが止める

人間が読むのは、定理の文と公理の一覧だけ

証明　約 5 万行

読まない。カーネルが読む

読む ⟶ 定理の文 ＋ 公理の一覧

発表 3 への橋

レビューが詰まるなら、
レビューする対象を小さくする

ただし、文が間違っていれば
間違ったことを正しく証明してしまう

「壁にあたったら、形式化のミスを疑う」

私がエージェントに渡した目標の一節（原文）

行き：Scala を「正」にして、Lean に写す



「戦略を Scala 版ベースに変更したはずやから、そこも確認してな」

9/15、私の発話（原文）

なぜ Scala か

- ADT と不変データ
- パターンマッチ
- 小さな純粋関数

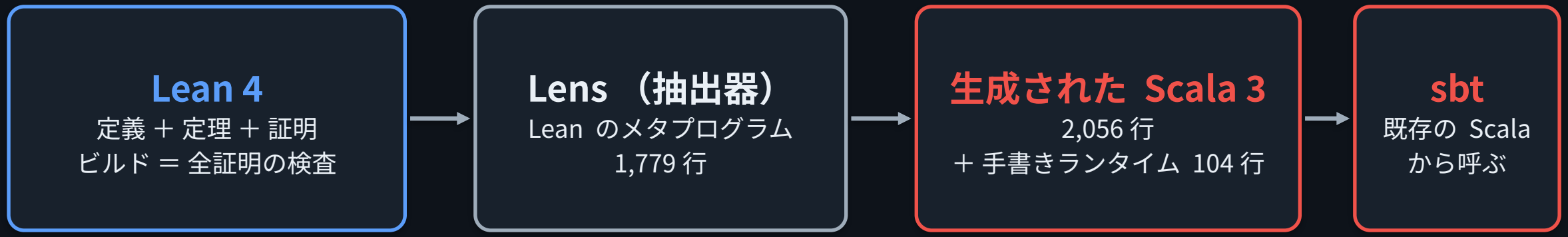
この書き方の Scala は、ほぼそのまま Lean になる

次の研究の指示書に書いた規則

Cats、ZIO、汎用 effect framework、過度な typeclass は導入しない

証明したい芯は、あえて素朴に

帰り：証明した Lean を、Scala 3 に抽出する



渡る	def と inductive （実行できる部分）
渡らない	theorem と Prop （証明は Lean に残る）

仕組み

Lean が再帰を内部の形に変える前の「等式」（f パターン ＝ 右辺）を取り出し、Scala の match に組み直す

Lean 4 から Scala 3 への抽出器は、調べた範囲では先行例が見つからなかった

※ 行き（未解決問題の証明）と帰り（lean4-peg ）は別のプロジェクト。同じコードで一周するのは、これから

inductive は、sealed trait と case class になる

Lean 4

```
inductive RBNode (α : Type) where
  | leaf
  | node (c : RBColor) (l : RBNode α)
         (k : String) (v : α)
         (r : RBNode α)
```

生成された Scala 3

```
sealed trait RBNode[A]
object RBNode {
  final case class leaf[A]() extends RBNode[A]
  final case class node[A](c: RBColor, l: RBNode[A],
      k: String, v: A, r: RBNode[A]) extends RBNode[A]
}
```

inductive → sealed trait + case class	**structure** → final case class
match → match	**Option / List** → Option / List
Except → Either	**Nat / Int / Char** → BigInt
自己末尾再帰 → @tailrec （269 中 16）	**theorem / Prop** → 消える

Scala 3 の enum にはならない。引数なしのコンストラクタも leaf() になる

関数は Scala に渡る。証明は Lean に残る

Lean 4 （定義と定理）

```
def stripPrefix? :
    List Char → List Char → Option (List Char)
  | [], rest => some rest
  | _ :: _, [] => none
  | c :: cs, d :: ds =>
      if beqChar c d then stripPrefix? cs ds
      else none

theorem stripPrefix?_suffix :
    ∀ (s input rest : List Char),
      stripPrefix? s input = some rest →
      ∃ p, input = p ++ rest
```

↑ 緑の定理と、その証明は Lean に残る

生成された Scala 3

```
@annotation.tailrec
def stripPrefix_u3f(x0: List[BigInt],
    x1: List[BigInt]): Option[List[BigInt]] =
  ((x0, x1) match {
    case (Nil, x2) => Some(x2)
    case ((head :: tail), Nil) => None
    case ((c :: cs), (d :: ds)) =>
      (if (beqChar(c, d) == true)
       then stripPrefix_u3f(cs, ds) else None)
  })
```

Scala 側に、証明の痕跡は無い。ただの関数に見える

でも「成功したら、入力は必ず p ++ rest の形」は証明済み

Char は BigInt （コードポイント）になる。読みやすさは、そこそこ

証明したもの、 Scala で動いているもの

PEG インタプリタ
健全・完全・決定的

型検査器
健全かつ完全

評価器
型が付けば詰まらない

コンパイラ
スタック VM が同じ値を返す

赤黒木
順序と平衡を保つ

JSON パーサ
RFC 8259 、往復定理

抽出した Scala を実行（2026-11-12）

```
$ sbt "shallotCli/run run ../examples/fact.shl"
ok:3628800
$ sbt "shallotCli/run run ../examples/collatz.shl"
ok:111
$ sbt "shallotCli/run eval \"1 + 2 * 3\""
ok:7
$ scripts/json-suite.sh
json-suite: OK (318 files, Lean ≡ Scala,
  y_/n_ all as required)
```

CLI の構文解析・型検査・評価は、すべて Lean から抽出したコード

リポジトリ全体： Lean 1,487 ファイル・約 32.7 万行、 sorry 0 （2026-11 時点）

証明が、60 件のテストが踏まなかった穴を見つけた

Shallot（証明つきの小さな言語）

```
def id ( x : int ) : int = x
( 1 + 2 )
```

→ parse-SyntaxError

```
def id ( x : int ) : int = x + 0
( 1 + 2 )
```

→ ok:3

何が起きているか

1. 構文解析器が x(1+2) を関数呼び出しとして読む
2. 関数の本体が、境界をまたいで次の式を飲み込む
3. 後ろに式が残らず、構文エラーになる

「印字して構文解析すると元に戻る」を証明しようとして、証明が通らない場所として見つかった。定理には、この形を除く条件を明示した

60 件の差分テストは、この形を一度も踏んでいなかった

正直な話：証明が守るのは Lean の中まで

証明済み
Lean の中

定義・定理・証明
カーネルが毎回検査
公理の検査 9,085 箇所

橋：突き合わせ

Lean の実行
＝ 期待値
＝ 抽出した Scala

60 ケース
JSON 318 本

信頼しているだけ
Lean の外

Lens 抽出器（未検証）
手書きランタイム 104 行
Scala コンパイラ、 JVM

抽出器のレビューで、変数捕獲のバグが 17 件見つかった（全部直して回帰テストにした）

差分テストは証明ではない。
保証は「抽出器が正しければ」

利点と欠点

利点

- **検査器は言いくるめられない**
  エージェントが嘘をついても、ビルドが落ちる
- **読む量が減る**
  証明ではなく、定理の文と公理の一覧を読む
- **テストが踏まない穴を見つける**
  境界条件を 1 件、証明が見つけた
- **出てくるのは普通の Scala**
  既存のコードから、そのまま呼べる

欠点

- **文が間違っていれば、間違いを証明する**
  形式化のミスは、人間が見るしかない
- **抽出器は未検証**
  変数捕獲のバグ 17 件を直した実績つき
- **生成コードに癖がある**
  数はすべて BigInt 、深い再帰に 256MB のスタック
- **渡せるのは純粋な部分だけ**
  partial や独自の型クラスは抽出できない
- **手間はゼロではない**
  19 日、 416 発話、引き継ぎ 約 8 回

全部は証明しない。間違えられない芯だけ証明する

普段の Scala （殻）

Cats Effect / ZIO 、 Play / Pekko 、 DB 、 HTTP 、画面

証明つきの芯

Lean で書いて証明し、
Scala に抽出した純粋関数

芯の候補

- 料金・按分・丸めの計算
- 権限の判定（発表 1 の続き）
- パーサ、バリデーション
- 状態遷移

ここは提案。私事の成果からの見立て

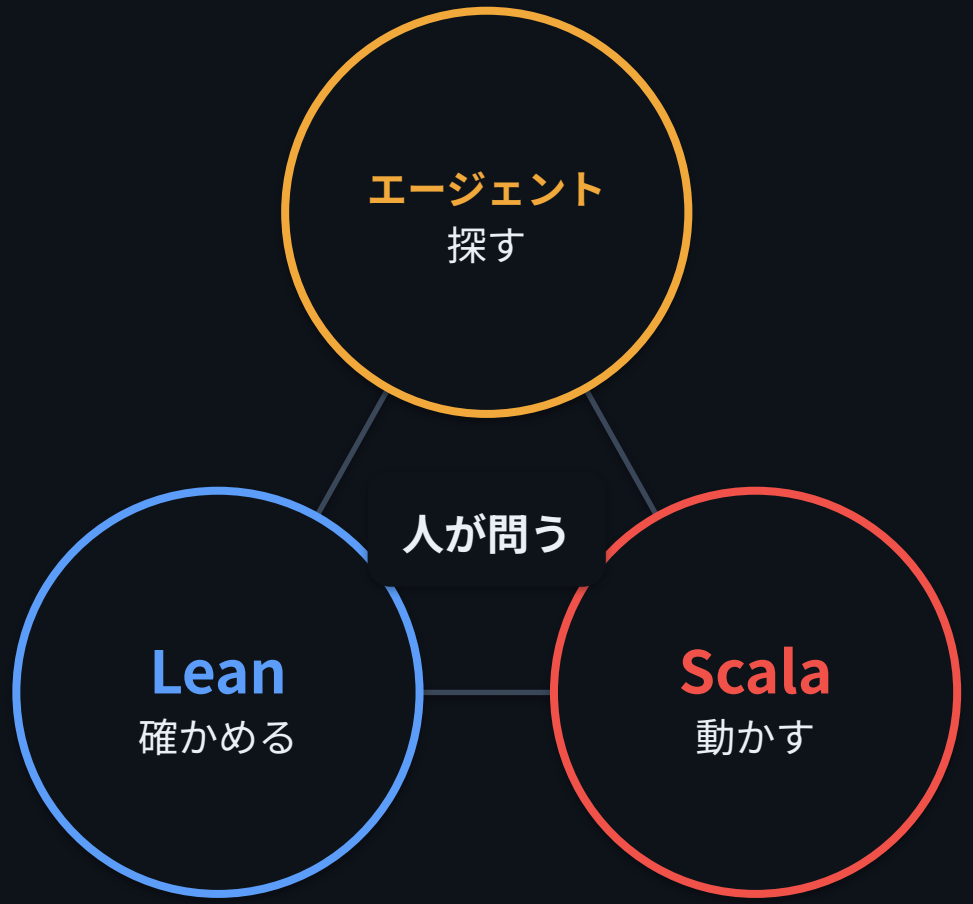
向かないもの： I/O が主役の所、画面、速さが命の内側ループ

エージェントが探し、 Lean が確かめ、 Scala が動かす

「あと俺にできるんは応援することだけや」

… ではなかった。私がやっていたのは

- 問いを立てる
- ズルを見張る
- 捨てる決断をする
- 定理の文を読む



型で嘘を防いできた Scala の人は、この世界のいちばん近くにいる

github.com/kmizu/lean4-peg